

DiT 在机器人动作生成中的完整讲解

从完整架构到训练流程与推理流程

技术讲解报告

2026 年 7 月 19 日

本文目标：以 GR00T N1 一类视觉-语言-动作系统为背景，系统解释用于动作生成的 DiT (Diffusion Transformer)。全文严格按照三个问题展开：

1. 完整的 DiT 架构包含哪些模块，各模块输入、输出和作用是什么；
2. 完整训练流程如何进行；
3. 完整推理流程如何进行。

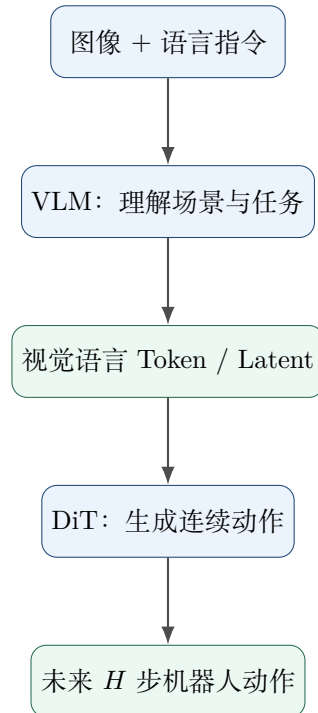
目录

1 整体认识：DiT 在 VLA 系统中的位置	3
2 符号与张量形状	3
2.1 机器人当前状态	3
2.2 动作块	4
2.3 VLM 条件 Token	4
2.4 Flow 时间	4
3 完整的 DiT 架构	4
3.1 整体结构图	5
3.2 模块一：Action Encoder	5
3.3 模块二：State Encoder	5
3.4 模块三：Time Encoder	6
3.5 模块四：VLM 条件投影	6
3.6 模块五：DiT Transformer Backbone	6
3.7 模块六：Action Decoder / Output Head	7
4 完整训练流程	8
4.1 步骤 1：读取专家轨迹样本	8
4.2 步骤 2：VLM 提取视觉语言条件	8
4.3 步骤 3：编码当前状态	8
4.4 步骤 4：采样随机噪声	8
4.5 步骤 5：采样 Flow 时间	8
4.6 步骤 6：构造中间带噪动作	9
4.7 步骤 7：构造监督速度	9

4.8	步骤 8: DiT 前向传播	9
4.9	步骤 9: 计算训练损失	9
4.10	步骤 10: 反向传播与参数更新	9
4.11	训练流程总图	10
5	完整推理流程	10
5.1	步骤 1: 获取当前观测	10
5.2	步骤 2: VLM 提取条件	10
5.3	步骤 3: 初始化随机动作块	10
5.4	步骤 4: 设置去噪/积分时间网格	11
5.5	步骤 5: 反复调用 DiT 预测速度	11
5.6	步骤 6: 数值积分更新动作	11
5.7	步骤 7: 得到最终动作块	11
5.8	步骤 8: 执行与重新规划	11
5.9	推理流程总图	12
6	训练与推理的核心区别	13
7	为什么 DiT 能把噪声变成动作	13
8	为什么不直接用 MLP 回归动作	13
9	与 LLM 语言输出的统一理解	14
10	最终总结	14

1 整体认识：DiT 在 VLA 系统中的位置

在视觉-语言-动作（Vision-Language-Action, VLA）系统中，可以把系统粗略分成两个层次：



VLM 主要负责回答：看到了什么、目标是什么、指令要求做什么、物体之间有哪些空间关系。
DiT 则负责回答：机器人在当前状态下，身体具体应该怎样连续运动。

一句话概括：VLM 决定“要做什么”，DiT 决定“身体具体怎么动”。

DiT 全称为 **Diffusion Transformer**。它并不等于“扩散过程”本身，而是扩散模型或 Flow Matching 中负责预测去噪方向的 Transformer 网络：

扩散/Flow Matching 规则 + Transformer 网络 = DiT 生成模型

其中：

- 扩散或 Flow Matching 规定如何构造带噪样本、模型应预测什么、损失怎样计算、推理时如何从噪声生成数据；
- Transformer 负责根据当前带噪动作和条件信息，预测下一步应如何修正动作。

2 符号与张量形状

2.1 机器人当前状态

记当前机器人状态为

$$s_t \in \mathbb{R}^{B \times d_s},$$

其中 B 是 Batch Size, d_s 是状态维度。状态可以包括关节角、关节速度、末端位姿、夹爪开合程度等。

2.2 动作块

模型一次预测未来 H 步动作：

$$A_t = [a_t, a_{t+1}, \dots, a_{t+H-1}],$$

其形状为

$$A_t \in \mathbb{R}^{B \times H \times d_a}.$$

这里：

- B ：一次处理多少条样本；
- H ：动作块长度或预测时域；
- d_a ：单步动作维度。

若 $B = 32$ 、 $H = 16$ 、 $d_a = 8$ ，则张量形状为 $[32, 16, 8]$ ，表示一次处理 32 条样本，每条样本含未来 16 步动作，每一步动作是 8 维。

2.3 VLM 条件 Token

VLM 输出一串视觉语言 Token：

$$C \in \mathbb{R}^{B \times N_c \times d_c},$$

其中 N_c 是 Token 数量， d_c 是 VLM 隐藏维度。这里使用的是 VLM 中间层隐藏表示，而不是已经经过 LM Head 的文字概率。

2.4 Flow 时间

记 Flow 时间为

$$\tau \in [0, 1].$$

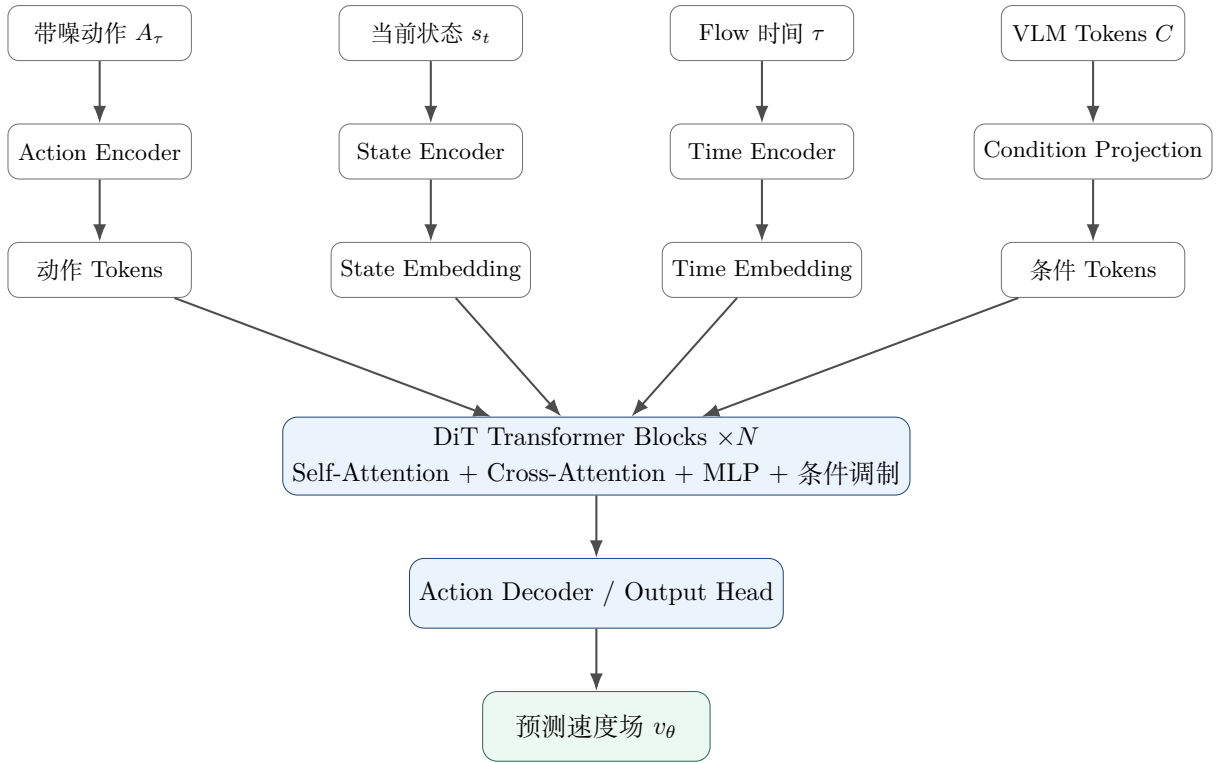
它表示当前动作样本处于“更接近纯噪声”还是“更接近真实动作”的阶段。

3 完整的 DiT 架构

一个用于机器人动作生成的完整 DiT 可以拆为六个核心模块：

1. Action Encoder；
2. State Encoder；
3. Time Encoder；
4. VLM 条件投影模块；
5. DiT Transformer Backbone；
6. Action Decoder / Output Head。

3.1 整体结构图



3.2 模块一：Action Encoder

输入：

$$A_\tau \in \mathbb{R}^{B \times H \times d_a}.$$

它是当前时刻的带噪动作块。

作用：把每一步低维机器人动作映射到 Transformer 的统一隐藏维度：

$$\mathbb{R}^{d_a} \rightarrow \mathbb{R}^D.$$

输出：

$$X_a \in \mathbb{R}^{B \times H \times D}.$$

每一个未来动作都变成一个动作 Token。

当前例子中, 若机器人每步动作为 7 个关节控制量加 1 个夹爪控制量, 则 $d_a = 8$ 。Action Encoder 可将 $[B, 16, 8]$ 映射为 $[B, 16, 1024]$ 。

3.3 模块二：State Encoder

输入：

$$s_t \in \mathbb{R}^{B \times d_s}.$$

作用：告诉 DiT 机器人当前的物理状态。相同任务在不同起始关节角或不同末端位置下，需要不同动作轨迹。

输出：

$$e_s \in \mathbb{R}^{B \times D}.$$

状态表示可作为状态 Token、与动作 Token 拼接、通过 Cross-Attention 注入，或用于条件归一化调制。

3.4 模块三：Time Encoder

输入： $\tau \in [0, 1]$ 。

作用： 告诉模型当前噪声阶段。相同数值在高噪声阶段可能需要大幅修正，在低噪声阶段则只需轻微修正。

典型结构为：

$$\tau \rightarrow \text{Fourier/Sinusoidal Encoding} \rightarrow \text{MLP} \rightarrow e_\tau.$$

输出：

$$e_\tau \in \mathbb{R}^{B \times D}.$$

时间表示通常通过 AdaLN (Adaptive LayerNorm)、加法、门控或额外条件 Token 注入每个 DiT Block。

3.5 模块四：VLM 条件投影

输入：

$$C \in \mathbb{R}^{B \times N_c \times d_c}.$$

作用： 把 VLM Token 映射到与 DiT 兼容的维度：

$$\mathbb{R}^{d_c} \rightarrow \mathbb{R}^D.$$

输出：

$$C' \in \mathbb{R}^{B \times N_c \times D}.$$

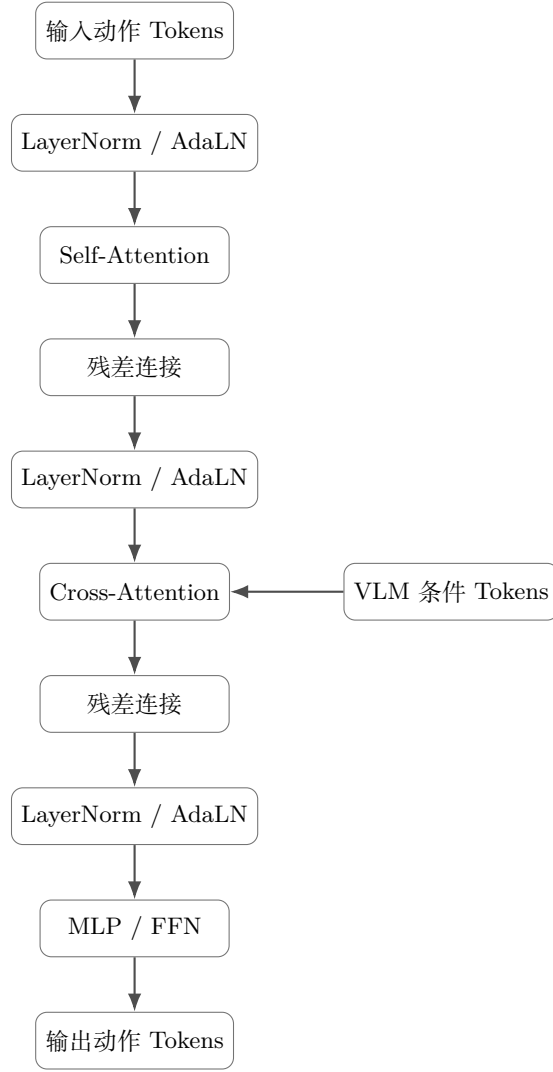
随后动作 Token 通过 Cross-Attention 读取这些视觉语言条件：

$$\text{Attention}(Q_{\text{action}}, K_{\text{VLM}}, V_{\text{VLM}}).$$

其中 Query 来自动作 Token，Key 和 Value 来自 VLM Token。它让每一步动作都能关注目标物体、指令语义和空间关系。

3.6 模块五：DiT Transformer Backbone

Backbone 由 N 个 DiT Block 堆叠构成。一个简化 Block 包含：



Self-Attention 建模动作序列内部的时间依赖。例如第 12 步动作必须知道前 11 步已把机械臂移动到哪里，才能保持轨迹连续并在正确时刻闭合夹爪。

Cross-Attention 让动作 Token 读取 VLM 条件，例如“红色杯子在哪里”“要抓取还是放置”“目标容器在哪一侧”。

MLP / FFN 对每个 Token 进行非线性变换，典型维度为 $D \rightarrow 4D \rightarrow D$ 。

残差连接 使深层 Transformer 更容易优化：

$$x_{l+1} = x_l + F(x_l).$$

3.7 模块六：Action Decoder / Output Head

输入：

$$X_{\text{out}} \in \mathbb{R}^{B \times H \times D}.$$

作用：将 Transformer 隐藏维度映射回动作维度：

$$\mathbb{R}^D \rightarrow \mathbb{R}^{d_a}.$$

输出：

$$\hat{V} \in \mathbb{R}^{B \times H \times d_a}.$$

在 Flow Matching 中，这通常不是最终动作，而是速度场：

$$v_\theta(A_\tau, \tau, s_t, C).$$

它表示当前动作样本应该向哪个方向移动，以及移动多快。

4 完整训练流程

一条训练样本写作：

$$(I_t, l, s_t, A_1),$$

其中 I_t 是图像， l 是语言指令， s_t 是当前状态， A_1 是真实未来动作块。

4.1 步骤 1：读取专家轨迹样本

例如：

- 图像：桌面上存在一个红色杯子；
- 指令：Pick up the red cup；
- 状态：当前关节角、速度和夹爪状态；
- 动作：未来 16 步专家动作。

真实动作形状为

$$A_1 \in \mathbb{R}^{B \times H \times d_a}.$$

4.2 步骤 2：VLM 提取视觉语言条件

图像与语言经过 VLM，取中间层视觉语言 Token：

$$C = \text{VLM}(I_t, l).$$

4.3 步骤 3：编码当前状态

$$e_s = \text{StateEncoder}(s_t).$$

4.4 步骤 4：采样随机噪声

采样与真实动作形状完全相同的高斯噪声：

$$A_0 \sim \mathcal{N}(0, I), \quad A_0 \in \mathbb{R}^{B \times H \times d_a}.$$

4.5 步骤 5：采样 Flow 时间

$$\tau \sim U(0, 1).$$

4.6 步骤 6: 构造中间带噪动作

在线性 Flow Matching 的简单形式中:

$$A_\tau = (1 - \tau)A_0 + \tau A_1.$$

当 $\tau = 0$ 时, $A_\tau = A_0$, 即纯噪声; 当 $\tau = 1$ 时, $A_\tau = A_1$, 即真实动作。

4.7 步骤 7: 构造监督速度

对上式关于 τ 求导:

$$\frac{dA_\tau}{d\tau} = A_1 - A_0.$$

因此真实速度目标为:

$$v^* = A_1 - A_0.$$

4.8 步骤 8: DiT 前向传播

模型接收四类输入:

$$(A_\tau, \tau, s_t, C),$$

输出预测速度:

$$\hat{v} = v_\theta(A_\tau, \tau, s_t, C).$$

4.9 步骤 9: 计算训练损失

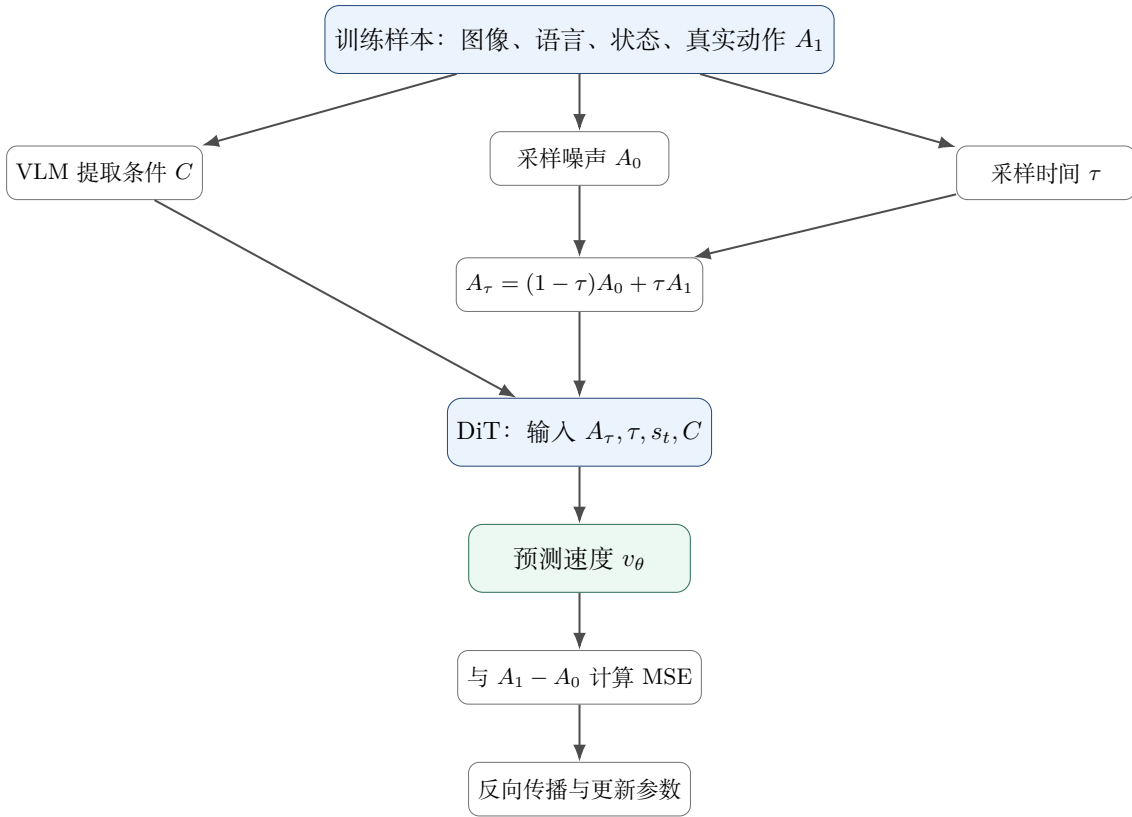
最常见的是均方误差:

$$\mathcal{L}_{\text{FM}} = \|v_\theta(A_\tau, \tau, s_t, C) - (A_1 - A_0)\|_2^2.$$

4.10 步骤 10: 反向传播与参数更新

根据损失反向传播, 更新 Action Encoder、State Encoder、Time Encoder、条件投影、DiT Backbone 和 Action Decoder。若采用端到端联合训练, 也可更新 VLM 的部分参数。

4.11 训练流程总图



5 完整推理流程

推理阶段与训练阶段的根本区别是：**没有真实未来动作 A_1** 。因此模型必须从随机动作噪声开始生成。

5.1 步骤 1：获取当前观测

机器人读取：

$$I_t, \quad l, \quad s_t.$$

即当前图像、语言指令与机器人状态。

5.2 步骤 2：VLM 提取条件

$$C = \text{VLM}(I_t, l).$$

5.3 步骤 3：初始化随机动作块

$$A^{(0)} \sim \mathcal{N}(0, I).$$

例如单条样本下：

$$A^{(0)} \in \mathbb{R}^{1 \times 16 \times 8}.$$

此时它是纯随机动作，不能直接执行。

5.4 步骤 4：设置去噪/积分时间网格

将 $[0, 1]$ 划分为 K 个步骤：

$$0 = \tau_0 < \tau_1 < \dots < \tau_K = 1.$$

5.5 步骤 5：反复调用 DiT 预测速度

在第 k 步：

$$v^{(k)} = v_\theta(A^{(k)}, \tau_k, s_t, C).$$

5.6 步骤 6：数值积分更新动作

最简单的 Euler 更新为：

$$A^{(k+1)} = A^{(k)} + \Delta\tau_k v^{(k)},$$

其中

$$\Delta\tau_k = \tau_{k+1} - \tau_k.$$

重复 K 次后，随机动作逐步变成合理动作。

5.7 步骤 7：得到最终动作块

$$\hat{A}_t = A^{(K)} = [\hat{a}_t, \hat{a}_{t+1}, \dots, \hat{a}_{t+H-1}].$$

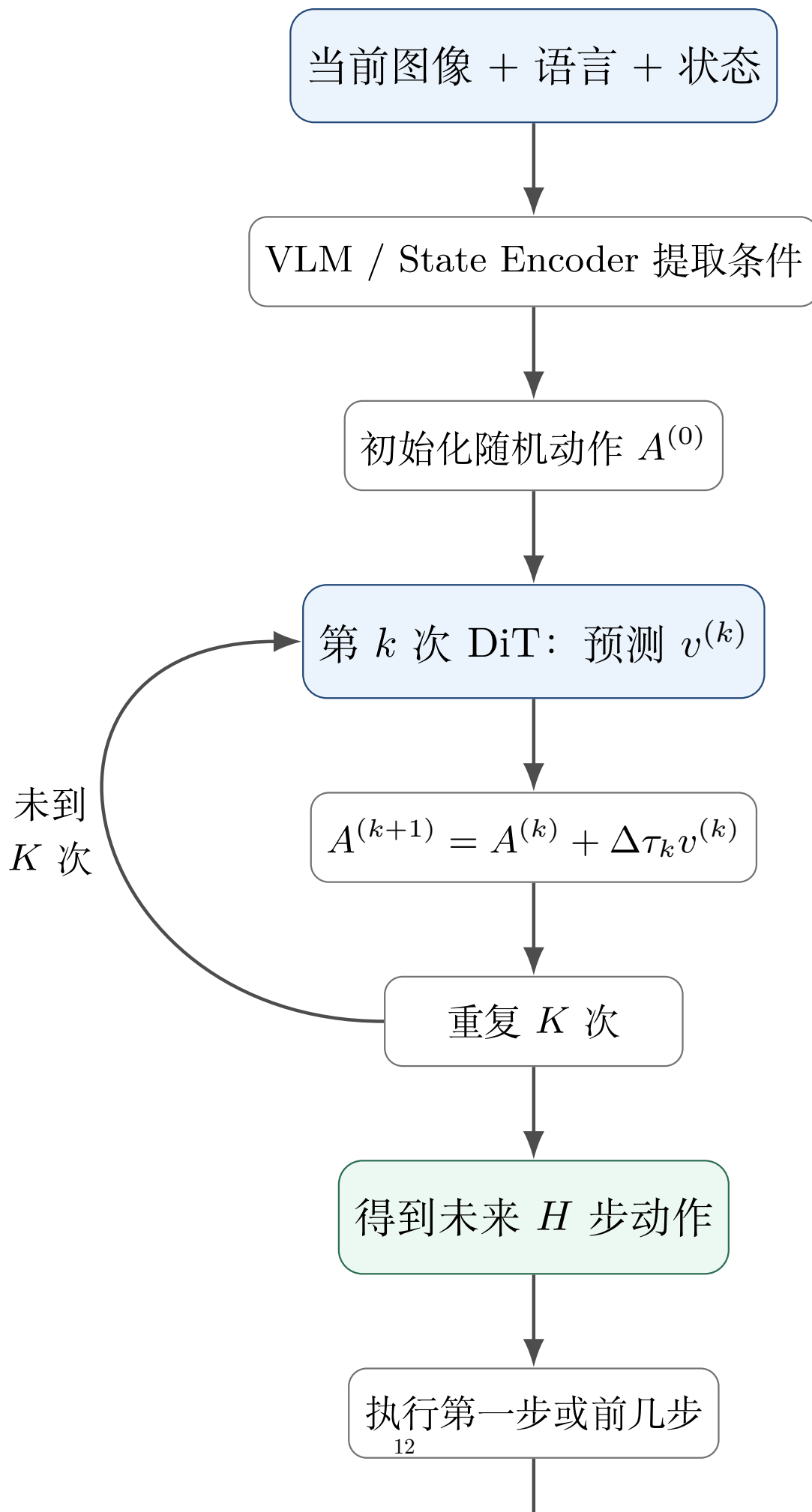
5.8 步骤 8：执行与重新规划

常见执行方式有三种：

1. 只执行第一步，然后重新感知并重新规划；
2. 执行前若干步，再重新规划；
3. 对连续多次预测做 Temporal Ensembling，以获得更平滑控制。

“预测未来 16 步”并不意味着机器人一定开环执行全部 16 步。实践中通常采用滚动时域控制：预测一段、执行一小段、重新观测、再次预测。

5.9 推理流程总图



6 训练与推理的核心区别

对比项	训练阶段	推理阶段
是否有真实动作	有真实未来动作 A_1	没有真实未来动作
初始动作	由噪声 A_0 与真实动作 A_1 插值得到 A_τ	直接从随机高斯噪声 $A^{(0)}$ 开始
模型目标	预测已知的真实速度 $A_1 - A_0$	预测当前速度并逐步更新动作
损失计算	使用 MSE 等监督损失	不计算训练损失
调用次数	每个训练样本通常随机取一个 τ 做一次前向传播	一次动作生成需要重复调用 DiT 多次
最终用途	更新网络参数	得到可执行动作块

7 为什么 DiT 能把噪声变成动作

DiT 并不是天然具有“去噪能力”。它在训练中反复看到如下对应关系：

带噪动作 + 时间 + 状态 + VLM 条件

正确速度方向 $A_1 - A_0$

经过大量训练，模型学习到一个条件速度场：

$$v_\theta(A, \tau, s, C).$$

这个速度场可以理解为：在动作空间的任意位置，给定当前机器人状态和任务条件，告诉当前动作样本下一步应该向哪里移动。推理时从随机点出发，只要不断沿速度场前进，就会逐渐靠近真实动作分布。

8 为什么不直接用 MLP 回归动作

可以直接训练：

$$\hat{A} = f(I, l, s),$$

但直接使用 MSE 回归时，若同一状态存在多种合理轨迹，模型容易将它们平均。例如抓杯子可以从左侧接近，也可以从右侧接近，平均后可能得到一条并不合理的中间轨迹。

Flow Matching 更接近学习条件动作分布：

$$p(A | I, l, s),$$

因此更适合表达多种合理动作、连续动作序列以及多模态轨迹分布。

9 与 LLM 语言输出的统一理解

LLM 的输出过程是：

Hidden State $\rightarrow$ LM Head $\rightarrow$ 词表 Logits $\rightarrow$ Token.

DiT 的输出过程是：

动作 Hidden Tokens $\rightarrow$ Action Head $\rightarrow$ 速度场 $\rightarrow$ 数值积分 $\rightarrow$ 动作序列.

二者都使用 Transformer 和输出 Head，但任务空间不同：LM Head 映射到词表，Action Head 映射到动作空间；LLM 自回归生成文字 Token，DiT 从噪声迭代生成动作。

10 最终总结

完整总结： 机器人动作 DiT 接收带噪动作块、Flow 时间、当前机器人状态和 VLM 视觉语言 Token，通过多层 Self-Attention、Cross-Attention、MLP 与条件调制，预测当前动作样本朝真实动作分布移动的速度方向。训练时利用真实动作和随机噪声构造监督速度场；推理时从高斯噪声出发，反复调用 DiT 并进行数值积分，最终生成未来一段连续机器人动作。机器人通常只执行其中第一步或前几步，再重新感知和规划，从而形成闭环控制。